

CMP5332

Object-Oriented Programming

Interfaces & Message Passing





Interfaces in Java



What is an Interface

In everyday language, the word “**interface**” means a **boundary** between two things, which may **interact** with each other across that boundary.

In **computer programming**, the word interface has at least **three** specific technical meanings:

- A **User Interface (UI)** is a boundary between a program and the human who uses it.
- An **Application Programming Interface (API)** is a boundary between one piece of code and another piece of code.
- In **OOP**, an interface is the public methods of a class that determines the boundaries of how users and applications can interact with object of that class.



What is an Interface

An interface is more than just a boundary between x and y — it defines the possible “**ways**” for x to **interact** with y :

- A **text-based user interface** has a set of **commands** which the user may enter via the keyboard; these are the only commands the program accepts from the user.
- A **Graphical User Interface (GUI)** has buttons, menus and other controls which the user may interact with; these are the only ways the user may interact with the program.
- An **Application Programming Interface (API)** supports a set of methods or requests, which may be invoked by other code; these are the only methods or requests supported.

Application Programming Interfaces (APIs)

According to the **principle of encapsulation**, classes interact with each other by calling public methods – **not by** accessing fields (which are private).

A class's public methods are its "API"; they define how the class may be used by other classes.

When we're thinking about the class's API, we only think about **what matters to the "outside world"** – that is, other classes which may call those methods:

- What public methods does the class have?
- What is the purpose of each method? Why should another class use it? What problem does it solve?
- What parameters does the method take?
- What does the method return?
- What exceptions can the method throw, in what circumstances?

All of these things define how the method **can be called**, and what the **result will be** – the caller needs to know them.



What is an API?

As long as the method does what it's supposed to do, **the caller doesn't need to know** how the method actually works. The body of the method — that is, the actual code executed when the method is called — is an implementation detail, **hidden** on the “other side” of the interface.

A car is a good analogy. When you're driving the **car**, you need to know that there is a **steering wheel**, an **accelerator pedal** and a **brake pedal**, and you need to know what will be the effect of turning the steering wheel or pressing the pedals down.

Of course, there are **physical mechanisms** inside the car to achieve those effects; but they are hidden, and irrelevant to the driver.

(Example) - AttendanceRegister API

For example, consider the following simple class, which models an attendance register:

```
import java.util.*;

public class AttendanceRegister {
    private final List<String> names = new ArrayList<>();

    public void addName(String name) throws AlreadyPresentException {
        if(names.contains(name)) {
            throw new AlreadyPresentException(name);
        }
        names.add(name);
    }

    public List<String> getNames() {
        return Collections.unmodifiableList(names);
    }
}
```

Note that this is exactly the same information which would appear in a **specification** for this class.

There is no need to specify how the methods should work, only how they should be called, and what their result should be.

This class's API has two methods:

- The **addName** method adds a name to the register. It takes a name as a parameter; an `AlreadyPresentException` is thrown if the name has already been added.
- The **getNames** method returns an unmodifiable list of names added to the register, in the order they were added.

Interfaces in OOP Languages

In Programming **an interface** defines what methods an object has, but not how these methods are implemented.

It **includes** the **name of the method**, its **parameters**, **return type**, and **what exceptions it can throw** — but **not the method's body**, which contains the instructions for executing that method.

That is, an interface contains only the **signature** of each method.

Example: declaring an interface this way in Java:

```
import java.util.List;

public interface Register {
    public void addName(String name) throws AlreadyPresentException;

    public List<String> getNames();
}
```

Register Interface

```
import java.util.List;

public interface Register {
    public void addName(String name) throws AlreadyPresentException;

    public List<String> getNames();
}
```

An interface is not the same as a class:

- It's declared as an **interface** instead of a class.
- Instead of **braces** containing a method body, there is a semicolon **;** after each method signature.
- An interface **cannot have instance fields**, since these are not part of how a class should be used by other classes.

Register Interface

```
import java.util.List;

public interface Register {
    public void addName(String name) throws AlreadyPresentException;

    public List<String> getNames();
}
```

- This interface **declares** that a Register can be used by calling its addName method or by calling its getName method.
- However, the interface doesn't say what these methods should do – it's up to the class to decide that.

Example: AttendanceRegister Class Implementing the Register Interface

The AttendanceRegister class has these methods with these signatures, but it also has **implementations** of the methods. Therefore we can say that the AttendanceRegister class “**implements**” the Register interface:

```
import java.util.*;

public class AttendanceRegister implements Register {
    private final List<String> names = new ArrayList<>();

    @Override
    public void addName(String name) throws AlreadyPresentException {
        if(names.contains(name)) {
            throw new AlreadyPresentException(name);
        }
        names.add(name);
    }

    @Override
    public List<String> getNames() {
        return Collections.unmodifiableList(names);
    }
}
```

AttendanceRegister Class

```
import java.util.*;

public class AttendanceRegister implements Register {
    private final List<String> names = new ArrayList<>();

    @Override
    public void addName(String name) throws AlreadyPresentException {
        if(names.contains(name)) {
            throw new AlreadyPresentException(name);
        }
        names.add(name);
    }

    @Override
    public List<String> getNames() {
        return Collections.unmodifiableList(names);
    }
}
```

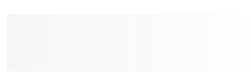
The **implements keyword** indicates that the `AttendanceRegister` class contains implementations for the methods in the `Register` interface.

The **@Override annotation** indicates the method “overrides” a method declared somewhere else (in this case, the `Register` interface) – this annotation is optional, but helps **make the code easier to read** and understand.

The **Register interface** defines what requests the class must accept and what kinds of response it can give; **AttendanceRegister** defines how those responses are computed.



Multiple Implementations



Although `Register` is the interface for the `AttendanceRegister` class, it is possible for **other classes** to have the same public methods with the same signatures.

In this case, those classes can also implement the `Register` interface.

Multiple Implementations

For example, the `SoloRegister` class models a register for a one-on-one tutorial which only one student is expected to attend:

```
import java.util.*;

public class SoloRegister implements Register {
    private String name = null;

    @Override
    public void addName(String name) throws AlreadyPresentException {
        if(this.name == null) {
            this.name = name;
        } else if(this.name.equals(name)) {
            throw new AlreadyPresentException(name);
        } else {
            String message = "Can't add second name: " + this.name + ", " + name;
            throw new IllegalStateException(message);
        }
    }

    @Override
    public List<String> getNames() {
        if(name == null) {
            return Collections.emptyList();
        } else {
            return Collections.singletonList(name);
        }
    }
}
```

The `SoloRegister` class is implemented quite differently to the `AttendanceRegister` class: since only one name needs to be stored, there is a field for a single name rather than a list of names.

Since the `getNames` method must return a list, this implementation uses methods from the `Collections` helper class to create one.

Interfaces are types

A Register can be used in two ways: either by calling its `addName` method, or by calling its `getNames` method. An `AttendanceRegister` can be used in both of those ways, so it is possible to use an `AttendanceRegister` as a `Register`:

```
Register register = new AttendanceRegister();
register.addName("Alice");
register.addName("Bob");
register.addName("Charles");
register.getNames().toString()
    "[Alice, Bob, Charles]" (String)
```

Similarly, a `SoloRegister` can be used as a `Register`:

```
Register solo = new SoloRegister();
solo.addName("Alice");
solo.getNames().toString()
    "[Alice]" (String)
```

In these examples, `Register` is the **type** of the variable, but the object is created using the `AttendanceRegister` constructor or the `SoloRegister` constructor. This works because both classes implement the `Register` interface.

Interfaces are types

We have already seen another situation where this is possible:

```
import java.util.*;  
List<String> names = new ArrayList<String>();
```

The variable's type is `List<String>`, but the object is created using the `ArrayList<String>` constructor. This works because `List` is an interface, and the `ArrayList` class implements this interface.

This makes sense: `List` is an [abstract data type](#), which means it specifies what methods a list has but not how those methods are implemented.

The `ArrayList` and `LinkedList` data structures are implementations of the `List` interface

Polymorphism

Since one method can have more than one implementation, when we write code to call this method, we don't necessarily know which implementation will be executed:

```
public class RegisterExample {  
    public static String getExampleNames(Register register) {  
        try {  
            register.addName("Alice");  
            register.addName("Bob");  
            register.addName("Charles");  
            return register.getNames().toString();  
        } catch(AlreadyPresentException ex) {  
            // should never happen; convert to unchecked exception  
            throw new RuntimeException(ex);  
        }  
    }  
}
```

This method takes a `Register` as its parameter, which means the method could receive either an `AttendanceRegister` object or a `SoloRegister` object.

Polymorphism

When we write this code, we know we will call the `addName` method on the `register` object. However, we don't know what class `register` is, so we don't know which implementation of the `addName` method will be executed.

This is an example of **polymorphism**: there is more than one class which the parameter `register` could have, and the call to the `addName` method executes different instructions depending on which class. The `getExampleNames` method is “polymorphic”.

Thinking about message passing, we know that `register` will accept the `addName` message, but we don't know how it will handle that message. Importantly, different implementations can result in different behaviour:

```
RegisterExample.getExampleNames(new AttendanceRegister())
    "[Alice, Bob, Charles]" (String)
RegisterExample.getExampleNames(new SoloRegister())
Exception: java.lang.IllegalStateException (Can't add second name: Alice, Bob)
    at SoloRegister.addName(SoloRegister.java:14)
```

The `getExampleNames` method has a different result the second time, where the `SoloRegister` class throws an exception but the `AttendanceRegister` class doesn't.